

Joining relational data

What happens to one row when we combine tables?

STAT 386
October 7, 2026

Review: row identity

talk_id	keyword	count
T1	faith	4
T1	prayer	2
T2	faith	1
T2	prayer	3

With a partner: What does one row represent? Which columns identify it?

What would go wrong if we pivoted using talk_id alone?

Review: the identifying columns

One row represents one talk-keyword combination.

The key is (talk_id, keyword). Each pair appears once.

```
wide = long.pivot(  
    index="talk_id", columns="keyword", values="count"  
)
```

talk_id	faith	prayer
T1	4	2
T2	1	3

A question that needs another table

How does average talk length differ between speaker groups?

talks: one row per talk

talk_id	speaker_id	num_words
T1	S1	1000
T2	S1	1200
T3	S1	1400
T4	S2	2400
T5	S3	1600

speakers: one row per speaker

speaker_id	speaker	group
S1	Avery	A
S2	Blair	B
S4	Casey	B

All names, groups, and values are fictional teaching data.

One master file repeats speaker information

A single CSV is convenient when someone has already prepared it for analysis.

talk_id	speaker_id	num_words	speaker	group
T1	S1	1000	Avery	A
T2	S1	1200	Avery	A
T3	S1	1400	Avery	A
T4	S2	2400	Blair	B

Avery's details repeat in three rows. A correction would require updating each copy.

If only one copy changes, the same speaker can have conflicting information.

Separate tables support reliable updates

A speaker table stores a speaker's details once. Each talk references the speaker ID.

A new speaker can enter the directory before giving a talk. A new talk can reference an existing speaker.

Many talks can share the same speaker, conference, and session records.

A join creates the combined table we need for analysis. We can export that result as a CSV.

Keys

Term	Meaning	Example
Primary key	Uniquely identifies a row in its table	talk_id in talks
Foreign key	References a key in another table	speaker_id in talks
Composite key	Uses several columns together	year + season

speaker_id repeats in talks and is unique in our clean speakers table.

CSV files do not enforce these relationships. We check them.

Cardinality depends on the join key

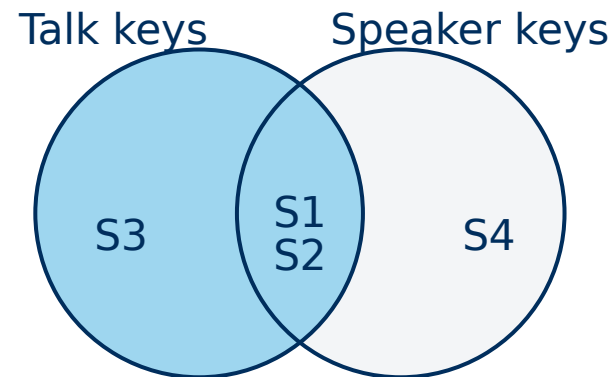
Relationship	Key uniqueness	Example
One-to-one	Unique on both sides	Talks + one transcript per talk
Many-to-one	Repeats left, unique right	Talks + speakers
One-to-many	Unique left, repeats right	Speakers + talks
Many-to-many	Repeats on both sides	Talks + multiple speaker records

For talks on the left and speakers on the right, we expect many-to-one.

Predict whether the intended row unit will survive the join.

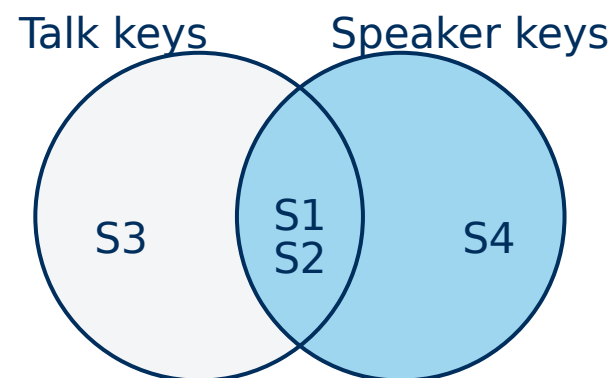
Join types as overlapping sets of keys

Left join



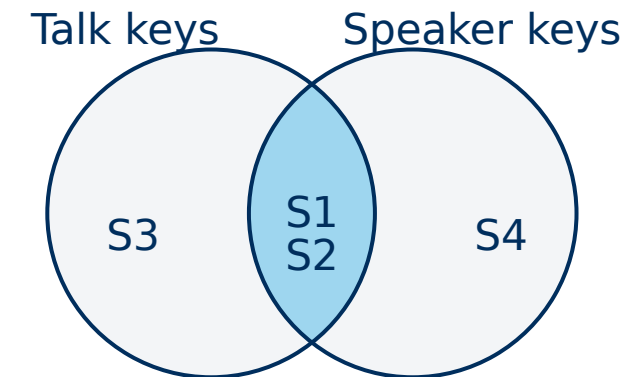
All talk keys: S1, S2, S3

Right join



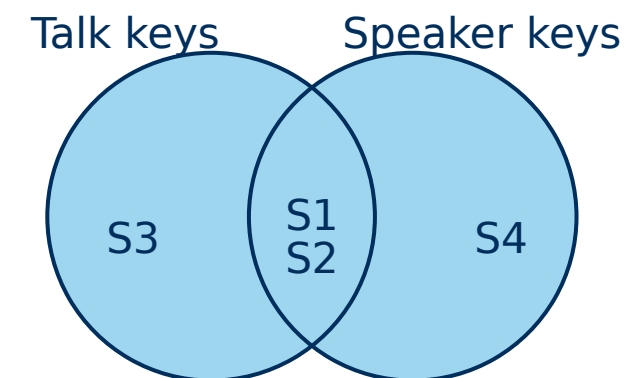
All speaker keys: S1, S2, S4

Inner join



Shared keys: S1, S2

Outer join



Either table: S1, S2, S3, S4

Blue regions show retained keys. Repeated keys can create multiple matching rows.

Prediction: a left join

Left: talks

talk_id	speaker_id	num_words
T1	S1	1000
T2	S1	1200
T3	S1	1400
T4	S2	2400
T5	S3	1600

Right: speakers

speaker_id	speaker	group
S1	Avery	A
S2	Blair	B
S4	Casey	B

We will match speaker_id and retain every talk.

How many rows? Which talk lacks speaker information? Does S4 appear?

Left join: every talk remains

```
left = pd.merge(
    talks, speakers, on="speaker_id", how="left"
)
left
```

talk_id	speaker_id	num_words	speaker	group
T1	S1	1000	Avery	A
T2	S1	1200	Avery	A
T3	S1	1400	Avery	A
T4	S2	2400	Blair	B
T5	S3	1600	NA	NA

5 talks remain. T5 has no matching speaker record.

Prediction: an inner join

talks

talk_id	speaker_id	num_words
T1	S1	1000
T2	S1	1200
T3	S1	1400
T4	S2	2400
T5	S3	1600

speakers

speaker_id	speaker	group
S1	Avery	A
S2	Blair	B
S4	Casey	B

We now retain only matching pairs.

Which talk disappears? How many rows remain? What population does the result describe?

Inner join: matched talks only

```
inner = pd.merge(
    talks, speakers, on="speaker_id", how="inner"
)
inner
```

talk_id	speaker_id	num_words	speaker	group
T1	S1	1000	Avery	A
T2	S1	1200	Avery	A
T3	S1	1400	Avery	A
T4	S2	2400	Blair	B

4 rows remain. T5 disappears because S3 has no speaker record.

The `_merge` column records where keys match

```
audited = pd.merge(
    talks, speakers, on="speaker_id", how="left",
    indicator=True,
)
audited[["talk_id", "speaker_id", "group", "_merge"]]
```

talk_id	speaker_id	group	_merge
T1	S1	A	both
T2	S1	A	both
T3	S1	A	both
T4	S2	B	both
T5	S3	NA	left_only

`indicator=True` adds `_merge`. Its values describe the source of each match.

The three matching labels

Value in _merge	Meaning
both	The key matched a row in each input table
left_only	The retained left row has no right-table match
right_only	The retained right row has no left-table match

Our left join contains both and left_only rows. It cannot contain right_only rows.

An outer or right join would retain S4 as right_only.

A row labeled both can still have missing metadata in another column.

Equality produces a mask, then loc keeps rows

```
mask = audited["_merge"].eq("left_only")
unmatched = audited.loc[mask]
unmatched[["talk_id", "speaker_id", "_merge"]]
```

talk_id	_merge	mask
T1	both	False
T2	both	False
T3	both	False
T4	both	False
T5	left_only	True

unmatched

talk_id	speaker_id	_merge
T5	S3	left_only

`.eq("left_only")` compares each value with the string "left_only". It is equivalent to `== "left_only"`.

`.loc[mask]` keeps the rows whose mask is True. Here, only T5 remains.

A summary after joining

```
summary = (  
    left.groupby("group", dropna=False, as_index=False)  
    .agg(n_talks=("talk_id", "size"),  
         mean_words=("num_words", "mean"))  
)
```

group	n_talks	mean_words
A	3	1200
B	1	2400
NA	1	1600

dropna=False retains the talk whose group is unknown.

These means weight talks equally. They do not weight speakers equally.

Prediction: a duplicated speaker key

Three talks for S1

talk_id	speaker_id	num_words
T1	S1	1000
T2	S1	1200
T3	S1	1400

Two speaker records for S1

speaker_id	speaker	group
S1	Avery	A
S1	Avery	B

We match speaker_id again. How many rows will S1 contribute?

How many rows will the full left join contain?

Each matching pair produces a row

```
bad = pd.merge(talks, speakers_conflicting,  
               on="speaker_id", how="left")  
bad[["talk_id", "speaker_id", "num_words", "group"]]
```

talk_id	speaker_id	num_words	group
T1	S1	1000	A
T1	S1	1000	B
T2	S1	1200	A
T2	S1	1200	B
T3	S1	1400	A
T3	S1	1400	B
T4	S2	2400	B
T5	S3	1600	NA

S1 contributes $3 \times 2 = 6$ pairs. The complete left join contains 8 rows.

Join size from key frequencies

For a key with m left rows and n right rows, an inner join produces $m \times n$ rows.

Left frequency m	Right frequency n	Matching pairs
3	1	3
3	2	6
3	0	0

A left join contributes $m \times \max(n, 1)$ rows for that key.

The unmatched S3 talk contributes one row to the left join.

Pair activity: enriching talks

Open `student_activity.py` in this repo and run its setup cell.

Goal: retain every talk and compare mean word count by speaker group.

1. Predict the relationship and the left-join row count.
2. Join the clean speaker table to talks.
3. Identify unmatched talks.
4. Summarize talk counts and mean word counts by group.

Use the first 7 minutes. Keep the unknown group visible.

Pair activity: an unexpected result

Run the supplied join with `speakers_conflicting`.

5. Explain the new row count using key frequencies.
6. Identify the talks that appear more than once.
7. Inspect the conflicting speaker records.
8. Propose a repair using the documented `record_status` rule.

Use the remaining 8 minutes. Prepare to explain why the repair is justified.

Activity debrief: the clean join

Check	Result
Input talks	12
Joined rows	12
Matched talks	11
Unmatched talk	A12, speaker S9

group	n_talks	mean_words
A	6	1500
B	5	1960
NA	1	2100

The group counts sum to all 12 talks.

Activity debrief: the extra rows

speaker_id	speaker	group	record_status
S1	Avery	A	current
S1	Avery	B	retired

S1 has 4 talks and 2 matching records, producing 8 rows.

The join contains 16 rows. A01, A02, A03, and A04 each appear twice.

The output mixes current and retired classifications for the same speaker.

Validation enforces the expected relationship

```
pd.merge(  
    activity_talks, speakers_conflicting,  
    on="speaker_id", how="left",  
    validate="many_to_one",  
)
```

MergeError: Merge keys are not unique in right dataset; not a many-to-one merge
many_to_one checks the right key. It permits repeated speaker IDs in talks.

Validation checks key uniqueness. It does not guarantee that every talk matches.

A repair based on the data meaning

```
current = speakers_conflicting.loc[
    speakers_conflicting["record_status"]
    .eq("current")
]
fixed = pd.merge(
    activity_talks, current,
    on="speaker_id", how="left",
    validate="many_to_one",
    indicator=True,
)
fixed[["talk_id", "group", "_merge"]]
```

talk_id	group	_merge
A01	A	both
A02	A	both
A03	A	both
A04	A	both
A05	B	both
A06	B	both
A07	B	both
A08	A	both
A09	A	both
A10	B	both
A11	B	both
A12	NA	left_only

The status rule restores 12 rows. A12 still lacks a speaker record.

Row multiplication changes the analysis

Return to the five-talk demonstration.

Quantity	Original talks	Join with conflicting speakers
Rows	5	8
Total word count	7,600	11,200
Mean word count	1,520	1,400

Duplicating S1 gives its shorter talks more weight.

A successful merge can still produce a misleading summary.

Prediction: a composite key

Talks (first three)

talk_id	year	season
A01	2025	April
A02	2025	April
A03	2025	October

conference_metadata

year	season	location
2025	April	North Hall
2025	October	South Hall
2026	April	East Hall
2026	October	West Hall

We want one conference location attached to each talk.

What happens if we join using year alone? Which columns identify a conference?

Composite key: year and season

```
with_location = pd.merge(
    activity_talks, conference_metadata,
    on=["year", "season"],
    how="left",
    validate="many_to_one",
)
with_location[["talk_id", "year",
               "season", "location"]]

```

talk_id	year	season	location
A01	2025	April	North Hall
A02	2025	April	North Hall
A03	2025	October	South Hall
A04	2025	October	South Hall
A05	2025	April	North Hall
A06	2025	October	South Hall
A07	2026	April	East Hall
A08	2026	April	East Hall
A09	2026	October	West Hall
A10	2026	April	East Hall
A11	2026	October	West Hall
A12	2026	October	West Hall

All 12 talks have one location. Joining on year alone would yield 24 rows.

A checking routine for joins

Before joining: describe the intended row unit and expected relationship.

```
talks["talk_id"].is_unique
speakers["speaker_id"].is_unique
speakers["speaker_id"].isna().sum()
```

Check	Output
Unique talk IDs	True
Unique speaker IDs	True
Missing speaker keys	0

During joining: specify keys, retention, and validate.

After joining: check row counts, row identity, and unmatched records.

Exit check

You have one row per enrollment with `course_id`.

The course catalog should have one row per `course_id`.

1. How do you retain every enrollment and attach the course title?
2. Which validation rule fits?
3. What could make the joined table longer?
4. How would you find enrollments without catalog matches?

Next time: What do missing values tell us about how the data were collected?

Exit check: one working answer

```
enriched = pd.merge(
    enrollments, catalog, on="course_id", how="left",
    validate="many_to_one", indicator=True,
)
unmatched = enriched.loc[
    enriched["_merge"].eq("left_only")
]
```

Duplicate matching course IDs in the catalog can repeat an enrollment.

The intended result keeps one row per enrollment.

Reference: names and overlapping columns

```
directory = speakers.rename(  
    columns={"speaker_id": "person_id"}  
)  
renamed = pd.merge(  
    talks, directory,  
    left_on="speaker_id",  
    right_on="person_id",  
    how="left",  
    validate="many_to_one",  
)  
renamed[["talk_id", "person_id",  
    "speaker"]]
```

talk_id	person_id	speaker
T1	S1	Avery
T2	S1	Avery
T3	S1	Avery
T4	S2	Blair
T5	NA	NA

left_on and right_on name each key. suffixes labels overlapping non-key columns.

Reference: missing keys

```
left_keys = pd.DataFrame({"key": [None], "x": [1]})  
right_keys = pd.DataFrame({"key": [None], "y": [2]})  
pd.merge(left_keys, right_keys, on="key", how="inner")
```

key	x	y
NA	1	2

pandas matches missing keys on both sides.

Check missing keys before relying on the match indicator.

Reference: joining and stacking

Goal	Operation
Attach related fields using keys	<code>pd.merge(...)</code>
Stack tables with the same row unit	<code>pd.concat([april, october], ignore_index=True)</code>

A join matches records. Concatenation stacks the rows supplied to it.

Before either operation, decide what one resulting row should represent.